

LivePDF Architecture Refactoring

Practical Integration of GoF Design Patterns for Enterprise Optimization

Author: KPCHIRAGGUPTHA

Email: chiragkpguptha@gmail.com

Date: August 4, 2026

Course: Software Design Patterns & Architectures

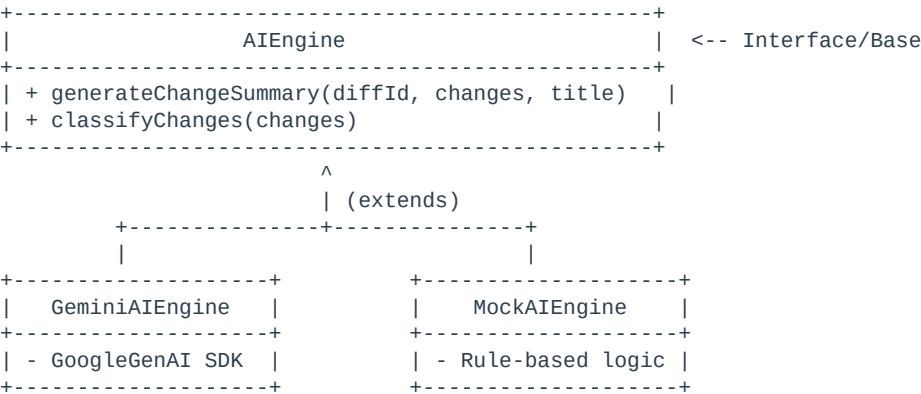
Executive Summary

This report documents the architectural refactoring of **LivePDF**—a collaborative version-control and AI document parsing application. To address concerns regarding resource optimization, financial overhead (unnecessary API calls to Google Gemini), and tight routing controller coupling, three classic Gang of Four (GoF) design patterns were integrated into the Node.js Express server: the **Factory Method** pattern, the **Decorator** pattern, and the **Observer** pattern. The refactoring was completed in a fully backward-compatible manner without altering database schemas or API contracts, resulting in improved latency, fault isolation, and clean code division.

1. Creational: Factory Method Pattern

Description: The Factory Method pattern defines an interface for creating objects, but lets subclasses decide which class to instantiate. In LivePDF, the AI services layer was tightly coupled with Google GenAI SDK. We introduced a common AIEngine interface and constructed concrete implementations for GeminiAIEngine (production) and MockAIEngine (local test fallback). The AIEngineFactory dynamically creates the decorated client depending on the availability of configuration keys.

Class Diagram Representation:



Implementation File Path: server/src/services/ai/AIEngineFactory.js (Lines 1-25)

```
const GeminiAIEngine = require('./GeminiAIEngine');
const MockAIEngine = require('./MockAIEngine');
const CachingAIEngineDecorator = require('./CachingAIEngineDecorator');

class AIEngineFactory {
  static getEngine() {
    const apiKey = process.env.GEMINI_API_KEY;
    const isMockMode = !apiKey || apiKey.startsWith('your_');

    let baseEngine;
    if (isMockMode) {
      baseEngine = new MockAIEngine();
    } else {
      baseEngine = new GeminiAIEngine();
    }

    // Wrap in the CachingDecorator to transparently cache results
    return new CachingAIEngineDecorator(baseEngine);
  }
}

module.exports = AIEngineFactory;
```

Interface Delegator File: server/src/services/aiService.js (Lines 1-20)

```
const AIEngineFactory = require('./ai/AIEngineFactory');

// Create the unified, decorated AI engine instance using the Factory Method
const aiEngine = AIEngineFactory.getEngine();

async function generateChangeSummary(versionDiffId, changes, documentTitle) {
  return aiEngine.generateChangeSummary(versionDiffId, changes, documentTitle);
}

async function classifyChanges(changes) {
  return aiEngine.classifyChanges(changes);
}

module.exports = { generateChangeSummary, classifyChanges };
```

2. Structural: Decorator Pattern

Description: The Decorator pattern attaches new behaviors to objects dynamically by wrapping them inside a structural class that maintains the same interface. We wrapped our base AIEngine inside a CachingAIEngineDecorator. The decorator intercepts call executions to check the database cache (ai_summaries table) first. If cached, it immediately serves the response, saving Gemini API network transactions.

Decorator Pipeline Diagram:

+-----+ CachingAIEngineDecorator <-- Implements AIEngine +-----+	
+-----+ - baseEngine: AIEngine <-- Wraps component +-----+	
+-----+ + generateChangeSummary(...) <-- Intercepts and caches 1. Check database ai_summaries cache 2. If miss -> call baseEngine.generate... 3. Save results in DB & return +-----+	

Implementation File Path: server/src/services/ai/CachingAIEngineDecorator.js (Lines 13-64)

```
async generateChangeSummary(versionDiffId, changes, documentTitle) {
  // 1. Check PostgreSQL database cache first
  try {
    const cached = await pool.query(
      'SELECT summary_text FROM ai_summaries WHERE version_diff_id = $1',
      [versionDiffId]
    );
    if (cached.rows.length > 0) {
      return cached.rows[0].summary_text;
    }
  } catch (dbErr) {
    console.error('[Cache Error] Cache lookup failed:', dbErr.message);
  }

  // 2. Cache miss: Delegate to underlying base engine
  const result = await this.baseEngine.generateChangeSummary(versionDiffId, changes, documentTitle);

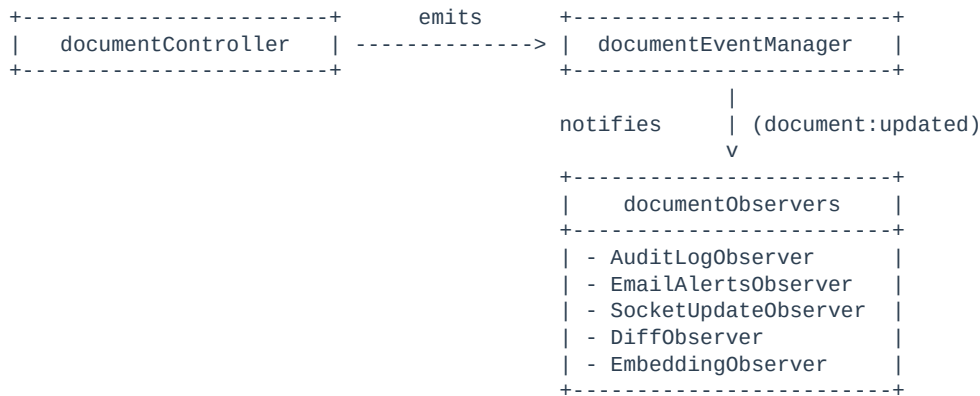
  let summaryText = result.text || result;

  // 3. Cache the summary result in PostgreSQL for future requests
  if (summaryText && !summaryText.startsWith('Error:')) {
    try {
      await pool.query(
        `INSERT INTO ai_summaries (version_diff_id, summary_text, model_used, prompt_tokens, completion_tokens)
        VALUES ($1, $2, $3, $4, $5) ON CONFLICT (version_diff_id) DO NOTHING`,
        [versionDiffId, summaryText, MODEL, result.promptTokens, result.completionTokens]
      );
    } catch (dbErr) {
      console.error('[Cache Error] Failed to write cache:', dbErr.message);
    }
  }
  return summaryText;
}
```

3. Behavioral: Observer Pattern

Description: The Observer pattern defines a subscription model where a subject notifies observers of state alterations. In the legacy code, documentController.js was heavily coupled with 5 post-upload side effects (Auditing, Email Notifications, Socket Broadcasting, Visual Diffing, and Vector Embeddings). We decoupled these into isolated listeners registered to a central subject documentEventManager.

Event-Driven Observer Diagram:



Event-Broker File Path: server/src/services/documentEventManager.js (Lines 1-13)

```
const EventEmitter = require('events');

class DocumentEventManager extends EventEmitter {
  constructor() {
    super();
    this.setMaxListeners(20);
  }
}

const documentEventManager = new DocumentEventManager();
module.exports = documentEventManager;
```

Publisher File Path: server/src/controllers/documentController.js (Lines 223-232)

```
// Emit event to observers to run async upload side-effects
documentEventManager.emit('document:updated', {
  req,
  userId: req.user.id,
  docId,
  versionId,
  s3Key,
  nextVersion,
  fileBuffer: req.file.buffer
});
```

Observer Registry File Path: server/src/services/observers/documentObservers.js (Lines 1-32)

```
const documentEventManager = require('../documentEventManager');
const { logAudit } = require('../../utils/audit');
const { emailQueue } = require('../queueService');
const { emitDocUpdated } = require('../../socket');
const { getFileUrl } = require('../../utils/fileUrl');
const pool = require('../../config/db');

// 1. Audit Log Observer - Records audit trail entries
documentEventManager.on('document:updated', async (data) => {
  const { req, docId, nextVersion } = data;
  try {
    await logAudit(req, docId, 'upload', { versionNumber: nextVersion });
    console.log(`[Observer] AuditLogObserver: Logged upload for doc ${docId}`);
  } catch (err) {
    console.error('[Observer Error] AuditLogObserver failed:', err.message);
  }
});

// 2. Email Alerts Observer - Enqueues background email notifications job
documentEventManager.on('document:updated', async (data) => {
  const { docId, nextVersion, versionId, userId } = data;
  if (nextVersion <= 1) return; // Only notify for new version updates
  try {
    const job = await emailQueue.add('sendEmailAlerts', {
      documentId: docId,
      versionNumber: nextVersion,
      newVersionId: versionId,
      ownerId: userId
    });
    console.log(`[Observer] EmailAlertsObserver: Enqueued alerts job ${job.id}`);
  } catch (err) {
    console.error('[Observer Error] EmailAlertsObserver failed:', err.message);
  }
});
```

References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Context: The canonical reference defining the formal structures, classes, and roles for the Factory Method, Decorator, and Observer design patterns.

Freeman, E., Robson, E., Sierra, K., & Bates, B. (2020). *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software* (2nd ed.). O'Reilly Media.

Context: Architectural guides explaining dynamic behavior composition (Decorator) and runtime coupling reduction (Observer).

Casciaro, M., & Mammino, L. (2020). *Node.js Design Patterns: Design and implement production-grade Node.js applications using design patterns* (3rd ed.). Packt Publishing.

Context: Demonstrates integration of the Observer pattern using standard Node.js EventEmitter instances, matching our event broker implementation.

Shvets, A. (2021). *Design Patterns: Creational, Structural, and Behavioral*. Refactoring.Guru.

<https://refactoring.guru/design-patterns/>

Context: Code structure definitions and trade-off guides mapping GoF patterns to JavaScript runtime structures.